Assignment 6

Q1. Kubernetes: Run Apache Pod

Create and run an Apache Pod.

Requirements:

1. Use image `httpd`.
2. Pod name: `apache-pod`
3. Verify the Pod using:

```
kubectl get pods
kubectl describe pod apache-pod
```

```
root@server-virtual-machine:~/Assignments# mkdir Assignment6 && cd Assignment6
root@server-virtual-machine:~/Assignments/Assignment6# kubectl run apache-pod --image=httpd
pod/apache-pod created
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get pod
NAME                          READY   STATUS    RESTARTS   AGE
apache-pod                    1/1     Running   0          19s
cdac-web-75687c58d8-bzf7f     1/1     Running   0          3h50m
cdac-web-75687c58d8-smtxm     1/1     Running   0          3h50m
cdac-web-75687c58d8-xsgkt     1/1     Running   0          3h50m
flask-app-6dffb545cb-59vvf    1/1     Running   0          3h34m
mysql-7975c6c656-r7thd        1/1     Running   0          3h41m
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl describe pod apache-pod
Name:             apache-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Fri, 30 Oct 2026 20:17:51 +0530
Labels:           run=apache-pod
Annotations:      <none>
Status:           Running
IP:               10.244.0.10
IPs:
  IP:  10.244.0.10
Containers:
  apache-pod:
    Container ID:   containerd://ba907a2eee9f71f14deac990ddfadc902d7244d456aaf41c3d1a4b3d817980f0
    Image:          httpd
    Image ID:       docker.io/library/httpd@sha256:d3accb516506c05ae0068bb54ba6798a10b249b64db91268fca692326cb02b4b
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Fri, 30 Oct 2026 20:18:05 +0530
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-7jxlt (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-7jxlt:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  42s   default-scheduler  Successfully assigned default/apache-pod to minikube
  Normal  Pulling    38s   kubelet            spec.containers{apache-pod}: Pulling image "httpd"
  Normal  Pulled     29s   kubelet            spec.containers{apache-pod}: Successfully pulled image "httpd" in 9.406s
bytes.
  Normal  Created    29s   kubelet            spec.containers{apache-pod}: Container created
  Normal  Started    28s   kubelet            spec.containers{apache-pod}: Container started
root@server-virtual-machine:~/Assignments/Assignment6# S
```

Q2. Kubernetes: Apache Deployment

Run Apache using a Kubernetes Deployment.

Requirements:

1. Create a deployment using image `httpd`.
2. Deployment name: `apache-deployment`
3. Verify using:

```
kubectl get deployments
kubectl get pods
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl create deployment apache-deployment --image=httpd
deployment.apps/apache-deployment created
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
apache-deployment   1/1     1            1           22s
cdac-web            3/3     3            3           3h59m
flask-app           1/1     1            1           3h45m
mysql               1/1     1            1           3h50m
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache-deployment-f7bc8694-qskl4    1/1     Running   0          38s
apache-pod                          1/1     Running   0          9m18s
cdac-web-75687c58d8-bzf7f           1/1     Running   0          3h59m
cdac-web-75687c58d8-smtxm           1/1     Running   0          3h59m
cdac-web-75687c58d8-xsgkt           1/1     Running   0          3h59m
flask-app-6dffb545cb-59vvf          1/1     Running   0          3h43m
mysql-7975c6c656-r7thd              1/1     Running   0          3h50m
root@server-virtual-machine:~/Assignments/Assignment6#
```

Q3. Kubernetes: Increase and Decrease Replicas

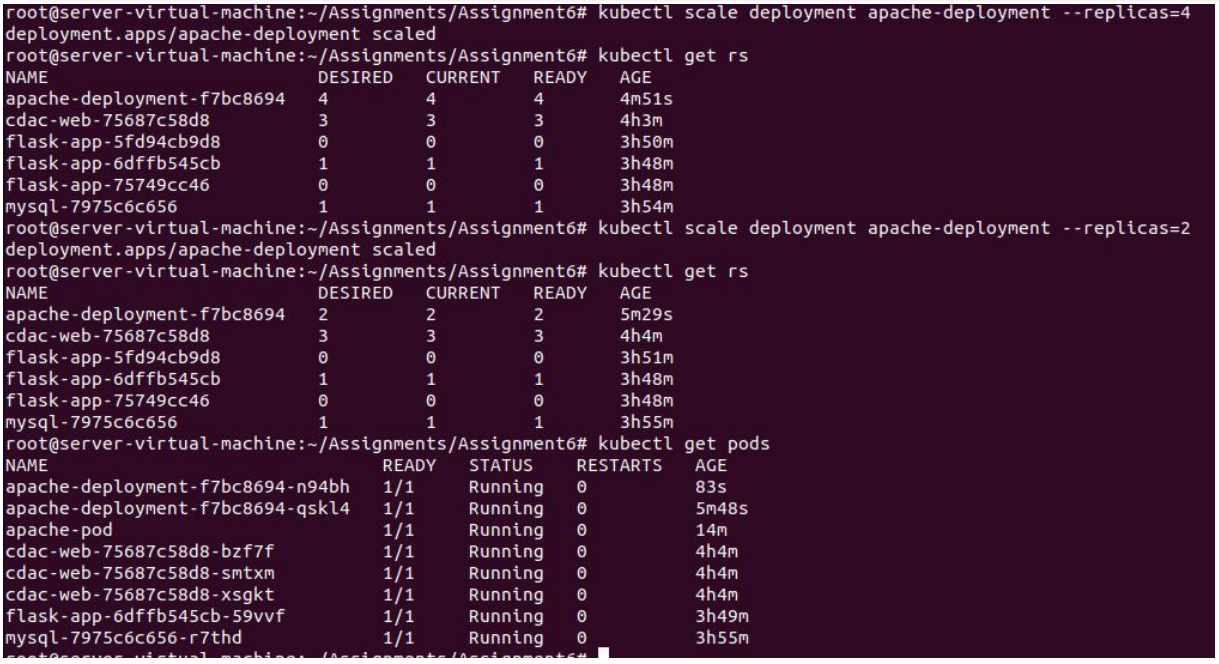
Scale the Apache Deployment.

Requirements:

1. Increase replicas to `4`.
2. Verify ReplicaSet and Pods.

3. Decrease replicas to `2`.
4. Verify changes using:

```
kubectl get rs
kubectl get pods
```



Q4. Kubernetes: Use YAML File

Create Apache Pod, Deployment, and Replica configuration using YAML.

Requirements:

1. Write a YAML file for Apache Deployment.
2. Use image `httpd`.
3. Set replicas to `3`.
4. Apply YAML file using:

```
kubectl apply -f apache-deployment.yaml
```

1. Verify all resources.

```
root@server-virtual-machine:~/Assignments/Assignment6# nano apache-deployment.yaml
  GNU nano 6.2                    apache-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-deployment
  labels:
    app: apache
spec:
  replicas: 3
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      labels:
        app: apache
    spec:
      containers:
      - name: apache
        image: httpd
        ports:
        - containerPort: 80
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl apply -f apache-deployment.yaml
deployment.apps/apache-deployment created
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
apache-deployment   3/3     3            3           53s
cdac-web            3/3     3            3           4h14m
flask-app           1/1     1            1           4h1m
mysql               1/1     1            1           4h5m
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache-deployment-5d7fb895f-c2dtz   1/1     Running   0          59s
apache-deployment-5d7fb895f-nclpr   1/1     Running   0          59s
apache-deployment-5d7fb895f-qg7h4   1/1     Running   0          59s
apache-pod                          1/1     Running   0          24m
cdac-web-75687c58d8-bzf7f           1/1     Running   0          4h14m
cdac-web-75687c58d8-smtxm           1/1     Running   0          4h14m
cdac-web-75687c58d8-xsgkt           1/1     Running   0          4h14m
flask-app-6dffb545cb-59vvf          1/1     Running   0          3h59m
mysql-7975c6c656-r7thd              1/1     Running   0          4h5m
root@server-virtual-machine:~/Assignments/Assignment6#
```

Q5. Kubernetes: ClusterIP Service

Expose Apache Deployment using ClusterIP.

Requirements:

1. Create a ClusterIP service.
2. Service name: `apache-clusterip`
3. Expose port `80`.
4. Test service access inside the cluster.

```
root@server-virtual-machine:~/Assignments/Assignment6#
root@server-virtual-machine:~/Assignments/Assignment6# kubectl expose deployment apache-deployment --type=ClusterIP --port=80 --name=apache-clusterip
service/apache-clusterip exposed
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get svc apache-clusterip
NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
apache-clusterip  ClusterIP   10.110.165.20   <none>        80/TCP    108s
root@server-virtual-machine:~/Assignments/Assignment6# kubectl run test-curl --image=curlimages/curl -i --tty --rm -- curl http://apache-clusterip:80
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test-curl, falling back to streaming logs: unable to upgrade connection: container test-curl not found in pod test-curl_default
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
pod "test-curl" deleted from default namespace
root@server-virtual-machine:~/Assignments/Assignment6#
```

Q6. Kubernetes: NodePort Service

Expose Apache Deployment using NodePort.

Requirements:

1. Create a NodePort service.
2. Service name: `apache-nodeport`

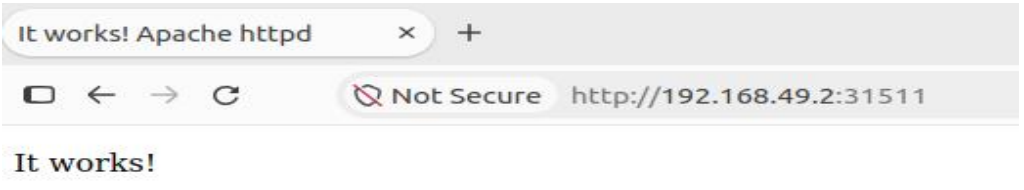
1. Update Kubernetes Deployment with new image.
2. Verify rolling update using:

```
kubectl rollout status deployment/flask-deployment
```

3. Access updated output in browser.
4. Expose port `80` .
5. Access Apache from browser using:

```
http://<node-ip>:<node-port>
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl expose deployment apache-deployment --type=NodePort --port=80 --name=apache-nodeport
service/apache-nodeport exposed
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get svc apache-nodeport
NAME              TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
apache-nodeport   NodePort   10.97.32.49    <none>        80:31511/TCP   29s
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION             CONTAINER-RUNTIME
minikube   Ready    control-plane   4h30m   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   6.8.0-138-generic (amd64)   containerd://2.3.4
root@server-virtual-machine:~/Assignments/Assignment6#
```



Q7. Kubernetes: LoadBalancer Service

Expose Apache Deployment using LoadBalancer.

Requirements:

1. Create a LoadBalancer service.
2. Service name: `apache-loadbalancer`
3. Expose port `80`.
4. Verify external IP using:

```
kubectl get svc
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl expose deployment apache-deployment --type=LoadBalancer --port=80 --name=apache-loadbalancer
service/apache-loadbalancer exposed
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get svc apache-loadbalancer
NAME                 TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
apache-loadbalancer   LoadBalancer   10.97.86.221   <pending>     80:32349/TCP   23s
```

```
server@server-virtual-machine:~$ sudo -i
[sudo] password for server:
root@server-virtual-machine:~# minikube tunnel
Status:
        machine: minikube
        pid: 41371
        route: 10.96.0.0/12 -> 192.168.49.2
        minikube: Running
        services: [apache-loadbalancer]
    errors:
                minikube: no errors
                router: no errors
                loadbalancer emulator: no errors
```

```
root@server-virtual-machine:~/Assignments/Assignment6# kubectl get svc apache-loadbalancer
NAME                 TYPE           CLUSTER-IP     EXTERNAL-IP    PORT(S)        AGE
apache-loadbalancer   LoadBalancer   10.97.86.221   10.97.86.221   80:32349/TCP   110s
root@server-virtual-machine:~/Assignments/Assignment6#
```

Q8. Kubernetes: Flask Application Deployment

Create a simple Python Flask application.

Requirements:

1. Create Flask app to display:
   - PRN number
   - Student name
2. Create Dockerfile.

3. Build Docker image.
4. Deploy Flask app as a Kubernetes Deployment.
5. Create a Service to expose the Flask app.
6. Access output in browser.

```
root@server-virtual-machine:~/Assignments/Assignment6# mkdir question8
root@server-virtual-machine:~/Assignments/Assignment6# cd question8
root@server-virtual-machine:~/Assignments/Assignment6/question8# nano app.py
root@server-virtual-machine:~/Assignments/Assignment6/question8# nano Dockerfile
```

```
  GNU nano 6.2                                  app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "PRN: 1234567890 <br> Name: John Doe"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
  GNU nano 6.2                                Dockerfile
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

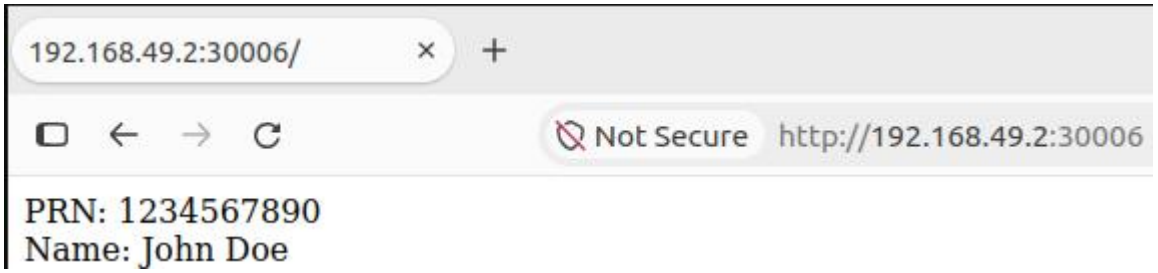
```
root@server-virtual-machine:~/Assignments/Assignment6/question8# docker build -t akshadasable/flask-app:v1 .
[+] Building 1.1s (9/9) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                       0.0s
 => => transferring dockerfile: 144B                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim         0.9s
 => [internal] load .dockerignore                                          0.0s
 => => transferring context: 2B                                            0.0s
 => [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd33   0.1s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd33   0.1s
 => [internal] load build context                                          0.0s
 => => transferring context: 28B                                           0.0s
 => CACHED [2/4] WORKDIR /app                                              0.0s
 => CACHED [3/4] RUN pip install flask                                     0.0s
 => CACHED [4/4] COPY app.py .                                             0.0s
 => exporting to image                                                     0.1s
 => => exporting layers                                                    0.0s
 => => exporting manifest sha256:38f286436b0a8285cb65aa5b684472d1a45cd25   0.0s
 => => exporting config sha256:6d85dc25eb09e2e98be3dc7c4b474fa9954ae7b04   0.0s
 => => exporting attestation manifest sha256:fbdfeda31ea5368d848f6692737   0.0s
 => => exporting manifest list sha256:d215326f633bfa1a210c85e065786f1bdb   0.0s
 => => naming to docker.io/akshadasable/flask-app:v1                       0.0s
 => => unpacking to docker.io/akshadasable/flask-app:v1                    0.0s
```

```
root@server-virtual-machine:~/Assignments/Assignment6/question8# docker push akshadasable/flask-app:v1
The push refers to repository [docker.io/akshadasable/flask-app]
b3ec39b36ae8: Pushed
44136fa355b3: Mounted from akshadasable/my_itiss_python
38513bd72563: Pushed
a1e22284271c: Pushed
fc7443084902: Pushed
36c28a308c6d: Pushed
ea56f685404a: Pushed
e0f10aec8203: Pushed
480ebc3351ed: Pushed
v1: digest: sha256:d215326f633bfa1a210c85e065786f1bdb9347fb97a48ac6f6e4851b1ca8b548 size: 856
```

```
root@server-virtual-machine:~/Assignments/Assignment6/question8# nano flask-deployment.yaml
root@server-virtual-machine:~/Assignments/Assignment6/question8# kubectl apply -f flask-deployment.yaml
deployment.apps/flask-deployment created
service/flask-service created
```

```
  GNU nano 6.2                    flask-deployment.yaml *
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-app
        image: your-dockerhub-username/flask-app:v1
        ports:
        - containerPort: 5000

---
```



Q9. Kubernetes: Rolling Update Flask Application

Perform rolling update on Flask application.
Requirements:

1. Modify Flask app output to include:
    - PRN number
    - Student name
    - Course name
2. Build new Docker image version.

```
root@server-virtual-machine:~/Assignments/Assignment6# mkdir question9 && cd question9
root@server-virtual-machine:~/Assignments/Assignment6/question9# nano app.py
```

```
  GNU nano 6.2                                app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "PRN: 1234567890 <br> Name: John Doe <br> Course: Cloud Computing"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
root@server-virtual-machine:~/Assignments/Assignment6/question9# docker build -t akshadasable/flask-app:v2 .
[+] Building 0.6s (9/9) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                        0.0s
 => => transferring dockerfile: 145B                                        0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim          0.4s
 => [internal] load .dockerignore                                           0.0s
 => => transferring context: 2B                                             0.0s
 => [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd33    0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd33    0.0s
 => [internal] load build context                                           0.0s
 => => transferring context: 28B                                            0.0s
 => CACHED [2/4] WORKDIR /app                                               0.0s
 => CACHED [3/4] RUN pip install flask                                      0.0s
 => CACHED [4/4] COPY app.py .                                              0.0s
 => exporting to image                                                      0.1s
 => => exporting layers                                                     0.0s
 => => exporting manifest sha256:179c42bdca693d5d123c3e3dca4de2d6844f199    0.0s
 => => exporting config sha256:995249961f4b61bb3eafcc5a4b08669a0df43f9f5    0.0s
 => => exporting attestation manifest sha256:31a583e65a323feee148e3c1f88    0.0s
 => => exporting manifest list sha256:4aca0d69d78f707f305eb36a91021c96b6    0.0s
 => => naming to docker.io/akshadasable/flask-app:v2                        0.0s
 => => unpacking to docker.io/akshadasable/flask-app:v2                     0.0s
```

```
root@server-virtual-machine:~/Assignments/Assignment6/question9# docker push akshadasable/flask-app:v2
The push refers to repository [docker.io/akshadasable/flask-app]
b3ec39b36ae8: Layer already exists
44136fa355b3: Already exists
e71811e540dd: Pushed
38513bd72563: Layer already exists
fc7443084902: Layer already exists
ea56f685404a: Layer already exists
36c28a308c6d: Layer already exists
e0f10aec8203: Layer already exists
aee3b87cbad8: Pushed
v2: digest: sha256:4aca0d69d78f707f305eb36a91021c96b67d9b8d2a5f57912084b559098f37a8 size: 856
```

```
root@server-virtual-machine:~/Assignments/Assignment6/question9# kubectl set image deployment/flask-deployment flask-app=akshadasable/flask-app:v2
deployment.apps/flask-deployment image updated
```

